

Quarantine

The offline clean room for untrusted model artifacts.

Every model you download is code you did not write. Quarantine runs it where it cannot hurt you, records everything it does, has a local open-weight model read that behaviour against the artifact's own declaration, blocks what it cannot explain, repairs what it can — and signs a receipt you can verify with OpenSSL.

an agent team, 3 of 6 using the model

contained execution

three outcomes

verified repair

signed receipts

no API key, no egress

Repository — <https://github.com/sushant-me/quarantine> (public, Apache-2.0)

Runs on — Python 3.12, Docker, and a local [qwen2.5-coder-3b](#) served by llama.cpp. No account, no paid API, no network at analysis time.

Status — working, measured, 75 tests, 60/60 eligibility checks.

This document describes the idea, how it works, and what has actually been measured — including the result that argues against us.

1. The problem

A model artifact is not data. A `pytorch_model.bin` is a pickle; a repository with `auto_map` in its config is a `modeling_*.py` that runs on your machine the moment the model is loaded. Loading it executes a stranger's code with your credentials, inside your network, on your laptop.

This is not hypothetical: pickles execute arbitrary code on unpickle by design, and the `trust_remote_code=True` flag exists precisely because model repositories ship custom Python that the framework is expected to run.

WHAT THE EXISTING SCANNERS DO

- Match global names against a denylist of dangerous functions, and read *pickle opcodes*.
- They are useful and fast — but **neither picklescan nor fickling opens a `.py` file**, so the code path is outside their input set entirely.
- They stop at a label. They cannot say what the artifact *did*, and they give you nothing to ship afterwards.

Measured on our labeled corpus (9 undeclared artifacts that all declare the same innocent API): picklescan **2/9**, fickling **2/9**. The two they catch are the two whose payload is a *pickle* — the only ones inside their input set. The other seven live in `custom_generate/generate.py` and `modeling_*.py`.

2. The idea

Judge what the artifact did, against what it declared — inside a box where doing it is harmless.

Three convictions hold the design together:

- **The box is the analysis.** Containment is not a safety net around the scan; executing the artifact is how you learn the truth. Behaviour cannot be evaded by renaming a function.
- **The AI decides meaning; code decides facts.** A model reads the evidence and reaches a verdict. Deterministic code decides what was observed, whether a claim is admissible, and whether a repair is correct — so no verdict rests on a model's opinion of its own work.
- **"We could not look" is a first-class outcome.** It is never reported as "fine". The first version of this tool did exactly that — it reported ALLOW for an artifact it had failed to run — and that defect is why the third outcome exists.

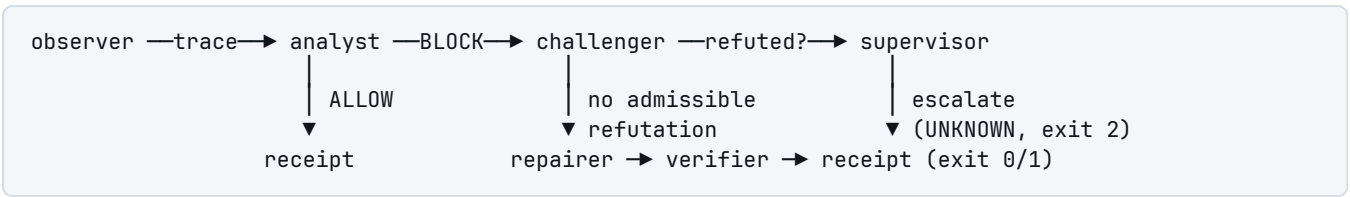
AN AGENT TEAM, NOT A PIPELINE

Six roles. **Three use the local model; three are deterministic code.** They never call each other: each posts to an append-only case file (`board.jsonl`) and a supervisor routes from what is on it. The whole transcript travels inside the signed receipt.

agent	kind	what it does
observer	deterministic	executes the artifact in Docker with the network off and records every sensitive operation via a CPython audit hook (PEP 578)
analyst	AI	reads the trace, the static capability graph and the declaration; returns ALLOW / BLOCK / UNKNOWN with evidence cited by trace id
challenger	AI	tries to <i>refute</i> the analyst — and may only succeed by quoting, verbatim, the sentence in the declaration that permits the behaviour
repairer	AI	writes the replacement function bodies; the harness writes the boilerplate, and an AST gate rejects any residual capability
verifier	deterministic	re-runs original and repair: identical outputs <i>and</i> zero capability operations
scribe	deterministic	assembles and signs the receipt, transcript included

Routing is a policy, not a prompt. Three branches exist purely to avoid the worst outcome for a security gate — saying "fine" when we did not look: nothing observed → escalate; the agents disagree and the challenger *grounded* its objection → escalate; the model was unreachable → escalate.

3. How it behaves



kind of artifact	verdict	exit	why
hides a DNS call on load	BLOCK	1	capability observed, and the declaration forbids it
benign control	ALLOW	0	looked, saw nothing capability-like
code path that cannot run	UNKNOWN	2	never observed → a human decides, never "fine"

The exit code is the integration point: `0` allow, `1` block, `2` escalate. **2 is not 0.**

THE REPAIR IS PROVEN, NOT ASSERTED

A block with no path forward is just a complaint. So the blocked artifact is repaired, and the repair is accepted only if it produces **identical outputs on 12 prompts** and performs **zero capability operations** while doing so — the second half measured from the trace, not read off the source. On the reference artifact the original performed 2 capability operations (`file.read /etc/hostname`, `socket.getaddrinfo`); the repair performed 0.

Across the corpus: 9/9 blocked · 7 repairs attempted · 7/7 capability-clean · 6/7 verified. The seventh is a refusal, not a failure — the original cannot be executed at all, so equivalence with it cannot be established, and the receipt says exactly that.

THE RECEIPT IS VERIFIABLE WITHOUT TRUSTING US

```
$ python tools/verify_receipt_standalone.py runs/probe/receipt.json \
    --pub runs/probe/keys/quarantine.pub.pem

ok    payload is canonical JSON (re-serialising reproduces the signed bytes)
ok    keyid matches the public key (ae3b68aa78ee693d)
ok    openssl: Signature Verified Successfully

    artifact  probe-custom-generate  verdict  BLOCK
VERIFIED — signed by the holder of this key, and the payload is unmodified.
```

That verifier imports nothing from this project and no third-party package, and checks the Ed25519 signature with **OpenSSL** — a different implementation than the one that signed it. Flip the verdict in the JSON from `BLOCK` to `ALLOW` and it reports **NOT VERIFIED**. Both directions were tested.

4. What has been measured

	Quarantine	pickle scan 1.0.5	fickling 0.1.12
our labeled corpus — 9 undeclared artifacts	9/9	2/9	2/9
4 benign controls	0 false positives	0	0
20 real published models from the Hub	20 allowed, 0 escalated	0 FP	18 FP (90%)
their corpus — 91 malicious samples, one per GHSA advisory	43 observed (47%)	88 (97%)	—
5 real Nepali/Indic models, incl. 2 requiring <code>trust_remote_code=True</code>	0 false positives	0	2 FP
deliberate sandbox escape, 10 primitives	0 escaped	—	—

The row that argues against us. On picklescan's *own* published corpus we observe 47% of the payloads and their denylist flags 97%. The reason is what each approach can see: their payloads mostly *name* a vulnerable entry point in a host library (`cProfile.run` , `pty.spawn` , `torch.utils.collect_env.run`) and call nothing when loaded in isolation — so there is no behaviour to observe, while a denylist reads the name and flags it anyway. Ours act on load.

So this is not a replacement for a pickle scanner. It is the other half of the pair — and the half that opens the `.py` files, and repairs what it blocks.

NEGATIVE RESULTS WE PUBLISHED RATHER THAN BURIED

- **No scanner bypass is claimed.** 19 one-pickle probes for standard-library callables that reach the network, the filesystem, a process or dynamic code: we looked for one that both incumbents miss and did not find it.
- **The torch analysis image measured worse and ships off by default.** It raised false positives from 0 to 2 of 5 on the Nepali/Indic controls, because importing a dependency performs `ctypes.dlopen` , sets its own environment variables and creates cache directories. A *measured* noise floor is subtracted; it fixes the plain-model case, not the framework one.

Twenty-nine defects were found by running the tool, and every one came from a control group, a third-party artifact, a rate or a corpus — never from the case being demonstrated. If you delete the AI calls from this codebase, what remains is a syscall trace and an AST listing: exactly the thing that already fails.

5. Pointed at Nepal

The challenge is about the Nepali open-source community, so the tool was pointed at it rather than described as if it applied.

- A search of **268 Nepali/Indic repositories** found **46 shipping `.py` files** and **15 declaring `auto_map`** — models that force `trust_remote_code=True`, i.e. running the author's Python on your machine. Neither incumbent scanner opens a single one of those files.
- Two real models joined the control set, including a **Nepali voice engine** (740 downloads) with a custom architecture. Running them **exposed two defects in our own tool** — a partial observation that was not escalated, and a `BLOCK` that could be grounded by citing an *error* rather than a capability.
- The first Nepali model we ran **escalated**, and the reason was the shape of the ecosystem: of the thirty most-downloaded Nepali models on the Hub, six are **GGUF** quantisations — the format most Nepali users actually download. Our reader did not know it. It does now.

What it costs us, stated plainly. Remote-code models are *escalated* by default, because their `modeling_*.py` needs torch and we will not approve or block a code path we never ran. The image that fixes it exists and measures worse. Proper event attribution is the work that closes this, and it is the clearest next step in the project.

6. Why it is a business, and what it is not

The constraint is the product. A local open-weight model with no API key, no vendor account and no bandwidth requirement is the only kind of artifact scanner that can be pointed at a customer's private model registry or run air-gapped — which is precisely where cloud-based scanners are structurally excluded. The free incumbents are static; hosting and containment, evidence an auditor will accept, and a repair you can ship are what a buyer pays for.

Beachhead: Nepal and South Asia. Closest design partners, cheapest go-to-market, and a deployment reality — intermittent bandwidth, data-residency rules — that this architecture already fits. The hook is the finding, not a pitch: a Nepali model author can be shown in one command whether their own artifact is checkable.

Intellectual property, honestly. The code is Apache-2.0 and is not the moat. The defensible assets are the labeled corpus and evaluation harness, the receipt format as an adopted standard, and the deterministic rule that binds an `ALLOW` to independently observed behaviour. Prior art is dense where a patent looks most tempting, and we say so.

WHAT WE ARE NOT CLAIMING

- **No proof the box cannot be broken.** Ten breakout primitives failed; that is a measurement, not a guarantee, and no kernel exploit was attempted.
- **Poisoned weights are out of scope.** We validate `safetensors` and `gguf` containers and read pickles — a code-execution question, not a values question.
- **The malicious corpus is ours**, which is why the third-party controls, the 20 real models, the Nepali models and the incumbent's own GHSA corpus exist — and why the 9/9 appears on the same page as the 47%.
- Nineteen limitations are listed in `LIMITATIONS.md`, including the ones that flatter the numbers least.

TRY IT IN THREE COMMANDS

```
git clone https://github.com/sushant-me/quarantine && cd quarantine
./scripts/serve_model.sh                # local open-weight model, no API key
python -m quarantine.cli inspect <artifact>  # 0 allow · 1 block · 2 escalate
```

Then verify a receipt without trusting any of it:

```
python tools/verify_receipt_standalone.py runs/probe/receipt.json \
--pub runs/probe/keys/quarantine.pub.pem  # openssl says Signature Verified
```

Quarantine — Frogtoberfest 2026 · `github.com/sushant-me/quarantine` · Apache-2.0 · 75 tests · 60/60 eligibility checks · presented with the network off by design.